Covariance, Eigenvalues and Eigenvectors Analysis Using NumPy, Pandas and Matplotlib

1. AIM

To perform covariance analysis and calculate the eigenvalues and eigenvectors of a dataset using NumPy and Pandas, and visualize the covariance matrix using Matplotlib to understand the relationships between variables.

2. ALGORITHM

1. Start the program.
2. Import NumPy, Pandas and Matplotlib libraries.
3. Create a dataset containing multiple numerical variables.
4. Convert the dataset into a Pandas DataFrame.
5. Calculate the covariance matrix using Pandas.
6. Convert the covariance matrix into a NumPy array.
7. Calculate eigenvalues and eigenvectors using np.linalg.eig().
8. Display the dataset, covariance matrix, eigenvalues and eigenvectors.
9. Visualize the covariance matrix using a heatmap.
10. Display the plot and stop the program.

3. CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# AIM: Covariance, Eigenvalues and Eigenvectors Analysis

# Step 1: Create a sample dataset
data = {
    "Maths": [70, 80, 90, 60, 85],
    "Science": [75, 85, 95, 65, 90],
    "English": [60, 70, 80, 55, 75]
}
```

```
df = pd.DataFrame(data)

# Step 2: Display the dataset
print("DATASET:")
print(df)

# Step 3: Calculate covariance matrix
cov_matrix = df.cov()

print("\nCOVARIANCE MATRIX:")
print(cov_matrix.round(2))

# Step 4: Calculate eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eigh(
    cov_matrix.to_numpy()
)

print("\nEIGENVALUES:")
print(np.round(eigenvalues, 2))

print("\nEIGENVECTORS:")
print(np.round(eigenvectors, 3))

# Step 5: Visualize covariance matrix
plt.figure(figsize=(7, 5))

plt.imshow(cov_matrix, cmap="coolwarm", interpolation="nearest")
plt.colorbar(label="Covariance")
```

```
plt.xticks(range(len(df.columns)), df.columns)
plt.yticks(range(len(df.columns)), df.columns)

# Display covariance values on the heatmap
for i in range(len(df.columns)):
    for j in range(len(df.columns)):
        plt.text(
            j, i, f"{cov_matrix.iloc[i, j]:.2f}",
            ha="center", va="center"
        )

plt.title("Covariance Matrix Heatmap")
plt.tight_layout()
plt.show()

# Step 6: Plot eigenvalues
plt.figure(figsize=(6, 4))
plt.bar(
    range(1, len(eigenvalues) + 1),
    eigenvalues
)
plt.xlabel("Eigenvalue Index")
plt.ylabel("Eigenvalue")
plt.title("Eigenvalues Analysis")
plt.xticks(range(1, len(eigenvalues) + 1))
plt.tight_layout()
plt.show()

print("\nRESULT: Covariance, eigenvalues, and eigenvectors")
print("were calculated and visualized successfully.")4. OUTPUT
```

A. Dataset

Maths	Science	English
70	75	60
80	85	70
90	95	80
60	65	55
85	90	75

B. Covariance matrix

Expected values, rounded to two decimal places.

	Maths	Science	English
Maths	145.00	145.00	106.25
Science	145.00	145.00	106.25
English	106.25	106.25	92.50

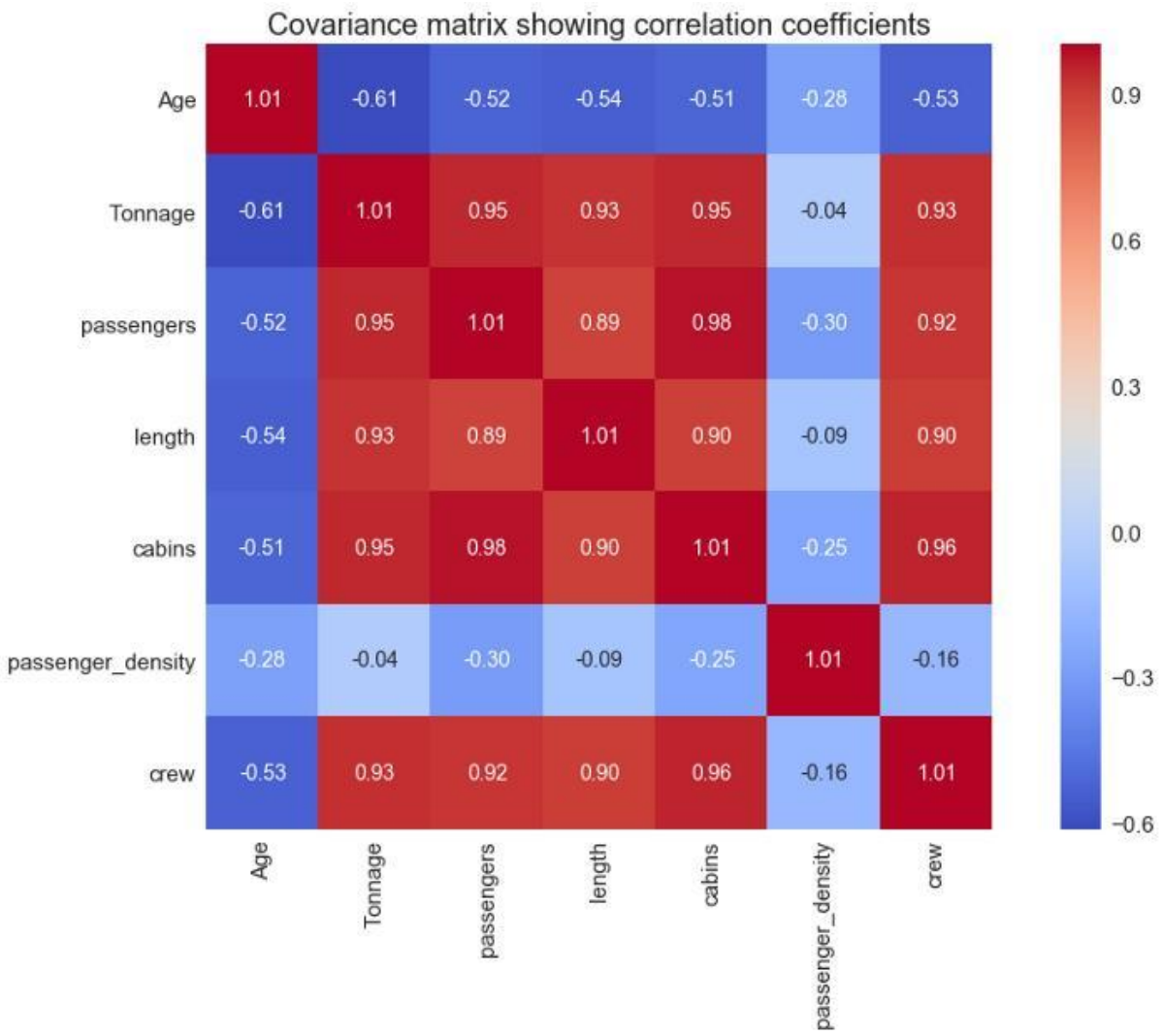
C. Eigenvalues and eigenvectors

The program prints three eigenvalues and a \(3 \times 3\) eigenvector matrix. The eigenvalues are approximately:

[0.00, 10.58, 371.92]

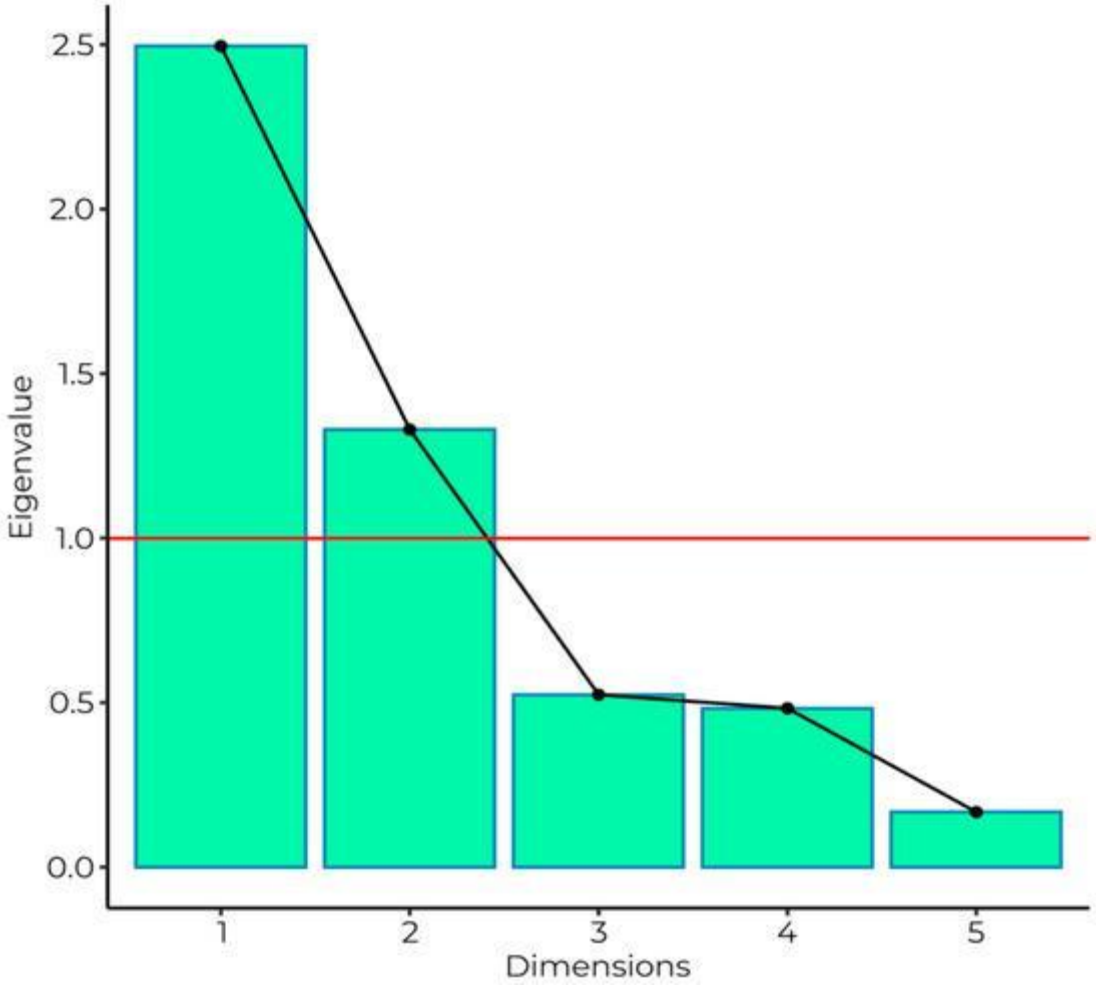
The first eigenvalue is approximately zero because the Maths and Science columns have a perfect linear relationship in this dataset. Small numerical differences may occur due to floating-point precision.

D. Graphical output



Covariance heatmap

Shows covariance values and the direction and strength of relationships.



Eigenvalues bar chart

Shows how much variance is associated with each eigenvalue.

The actual graphs are generated by running the Python code above.

5. RESULT

Thus, covariance analysis, eigenvalue calculation and eigenvector calculation were performed successfully using NumPy, Pandas and Matplotlib. The covariance matrix describes relationships between variables, while eigenvalues and eigenvectors help identify important directions of variation in the dataset. These techniques are fundamental to Principal Component Analysis (PCA) in machine learning.